Labsheet 9 : BTCS301P-AS09

Computer Vision Lab

Khushi Choudhary CU240251564

In [2]:
```
# STEP 1: Install and Import Required Libraries
import cv2
import torch
import time
import urllib.request
import numpy as np
import matplotlib.pyplot as plt
from ultralytics import YOLO
print(f"PyTorch Version: {torch.__version__}")
print(f"CUDA Hardware Acceleration Available: {torch.cuda.is_available()}\n")
```

```
PyTorch Version: 2.14.0+cpu
CUDA Hardware Acceleration Available: False
```

In [ ]:
```
# STEP 2: Load Pre-trained Deep Learning Models
print("Loading pre-trained YOLOv8 models...")
model_nano = YOLO('yolov8n.pt')
model_small = YOLO('yolov8s.pt')
print("Models loaded successfully.\n")
```

```
Loading pre-trained YOLOv8 models...
Models loaded successfully.
```

In [4]:
```
# STEP 3: Direct Download of ONE Sample Standard Dataset Image
image_url = "https://raw.githubusercontent.com/ultralytics/ultralytics/main/ultralytics/assets/bus.jpg"
image_path = "coco_sample_image.jpg"
print("Downloading standard sample image from MS COCO dataset...")
urllib.request.urlretrieve(image_url, image_path)
print("Image downloaded successfully!\n")
```

```
Downloading standard sample image from MS COCO dataset...
Image downloaded successfully!
```

In [ ]:
```
# STEP 4: Image Preprocessing Pipeline
raw_img = cv2.imread(image_path)
resized_img = cv2.resize(raw_img, (640, 640))
rgb_img = cv2.cvtColor(resized_img, cv2.COLOR_BGR2RGB)
normalized_img = rgb_img / 255.0
print("--- Preprocessing Verification ---")
print(f"Raw Image Shape: {raw_img.shape}")
print(f"Resized & Normalized Image Tensor Shape: {normalized_img.shape}")
print(f"Pixel Intensity Range: [{normalized_img.min():.1f}, {normalized_img.max():.1f}]\n")

plt.figure(figsize=(4, 4))
plt.imshow(rgb_img)
plt.title("Preprocessed Input Image")
plt.axis('off')
plt.show()
```

```
--- Preprocessing Verification ---
Raw Image Shape: (1080, 810, 3)
Resized & Normalized Image Tensor Shape: (640, 640, 3)
Pixel Intensity Range: [0.0, 1.0]
```



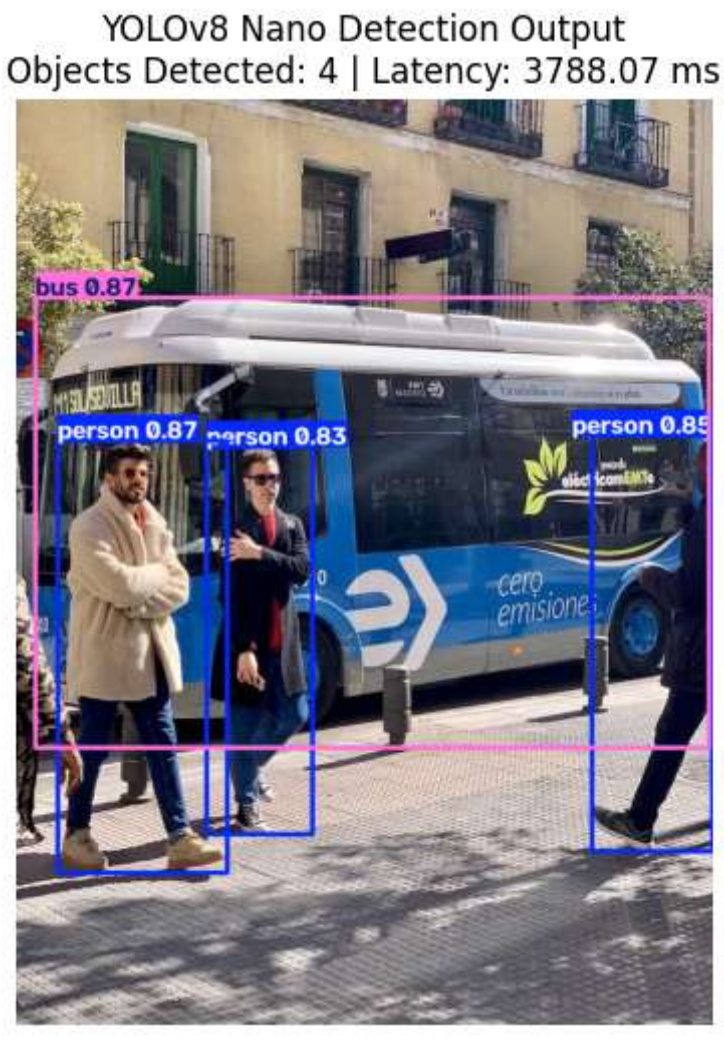
In [6]:
```
# STEPS 5 & 6: Object Detection, Localization, and Visualization
def run_object_detection(model, img_path, confidence_threshold=0.5):
```

```
    start_time = time.time()
    # Model inference (handles internal tensor operations)
    results = model.predict(source=img_path, conf=confidence_threshold, verbose=False)
    inference_latency = (time.time() - start_time) * 1000  # Latency in ms
    # Process output tensor data
    detection_result = results[0]
    bounding_boxes = detection_result.boxes
    # Render bounding boxes, class names, and confidence scores onto frame
    annotated_bgr = detection_result.plot()
    annotated_rgb = cv2.cvtColor(annotated_bgr, cv2.COLOR_BGR2RGB)

    print(f"=== Detection Output: {model.model_name} ===")
    print(f"Inference Latency: {inference_latency:.2f} ms")
    print(f"Total Objects Localized: {len(bounding_boxes)}")
    print("-" * 65)
    print(f"{'Class Name':<15} | {'Confidence Score':<18} | {'Bounding Box [xmin, ymin, xmax, ymax]'}")
    print("-" * 65)

    for box in bounding_boxes:
        class_id = int(box.cls[0])
        class_label = model.names[class_id]
        confidence = float(box.conf[0])
        coordinates = [round(c, 1) for c in box.xyxy[0].tolist()]
        print(f"{class_label:<15} | {confidence:<18.2f} | {coordinates}")
    print("-" * 65 + "\n")
    return annotated_rgb, inference_latency, len(bounding_boxes)
# Execute detection using YOLOv8 Nano
annotated_nano, time_nano, count_nano = run_object_detection(model_nano, image_path)
# Display Visual Detections
plt.figure(figsize=(8, 6))
plt.imshow(annotated_nano)
plt.title(f"YOLOv8 Nano Detection Output\nObjects Detected: {count_nano} | Latency: {time_nano:.2f} ms")
plt.axis('off')
plt.show()
```

```
=== Detection Output: yolov8n.pt ===
Inference Latency: 3788.07 ms
Total Objects Localized: 4
-----------------------------------------------------------------
Class Name      | Confidence Score   | Bounding Box [xmin, ymin, xmax, ymax]
-----------------------------------------------------------------
bus             | 0.87               | [22.9, 231.3, 805.0, 756.8]
person          | 0.87               | [48.6, 398.6, 245.3, 902.7]
person          | 0.85               | [669.5, 392.2, 809.7, 877.0]
person          | 0.83               | [221.5, 405.8, 345.0, 857.5]
-----------------------------------------------------------------
```



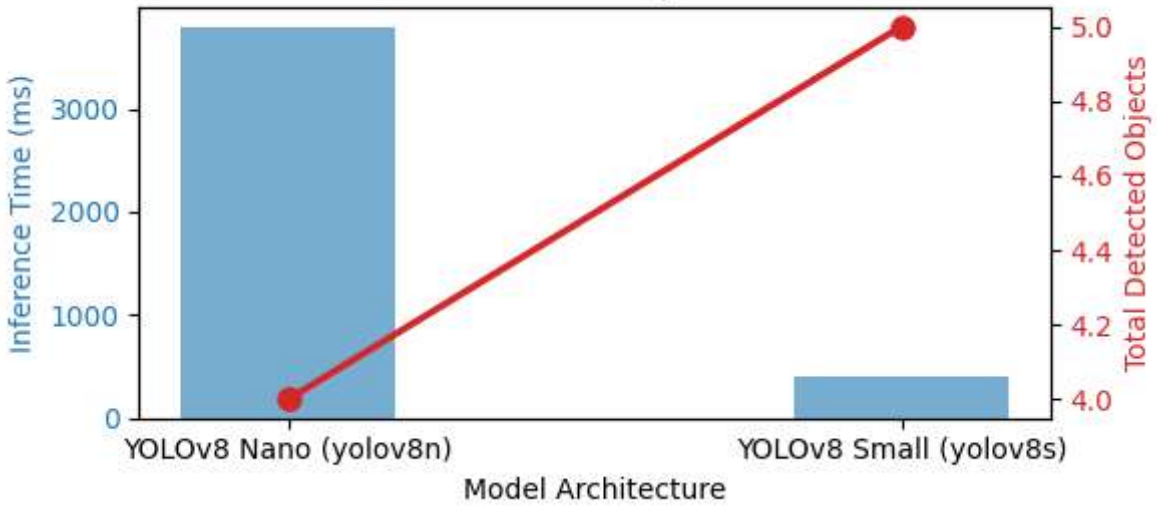
In [ ]:
```
# STEPS 7 & 8: Model Evaluation & Performance Comparison
annotated_small, time_small, count_small = run_object_detection(model_small, image_path)
model_names = ['YOLOv8 Nano (yolov8n)', 'YOLOv8 Small (yolov8s)']
latency_list = [time_nano, time_small]
detection_counts = [count_nano, count_small]
fig, ax1 = plt.subplots(figsize=(6, 3))

color = 'tab:blue'
ax1.set_xlabel('Model Architecture')
ax1.set_ylabel('Inference Time (ms)', color=color)
```

```
bars = ax1.bar(model_names, latency_list, color=color, alpha=0.6, width=0.35)
ax1.tick_params(axis='y', labelcolor=color)
ax2 = ax1.twinx()
color = 'tab:red'
ax2.set_ylabel('Total Detected Objects', color=color)
lines = ax2.plot(model_names, detection_counts, color=color, marker='o', linewidth=2.5, markersize=8)
ax2.tick_params(axis='y', labelcolor=color)

plt.title("Model Benchmark – Inference Speed vs Detection Count")
fig.tight_layout()
plt.show()
```

```
=== Detection Output: yolov8s.pt ===
Inference Latency: 399.48 ms
Total Objects Localized: 5
------------------------------------------------------------------
Class Name      | Confidence Score   | Bounding Box [xmin, ymin, xmax, ymax]
------------------------------------------------------------------
bus             | 0.92               | [15.0, 228.5, 808.2, 748.1]
person          | 0.89               | [667.3, 389.6, 809.7, 878.5]
person          | 0.89               | [49.9, 401.8, 244.5, 901.8]
person          | 0.88               | [222.1, 408.2, 345.5, 860.7]
person          | 0.61               | [0.2, 550.8, 65.2, 869.0]
------------------------------------------------------------------
```



Step 9: Real-World Application Analysis

Application Area	Suitable Model Architecture	Key Justification & Operational Requirements
Autonomous Driving	**YOLOv8 Nano / SSD**	Requires real-time frame rates (≥ 30 FPS) and extremely low latency (< 15 ms) to identify pedestrians, vehicles, lane boundaries, and traffic signs instantly. Safety depends on zero frame delays.
Surveillance & Smart Cities	**YOLOv8 Small / Edge Models**	Balances high spatial localization accuracy for crowd monitoring, vehicle tracking, and intrusion detection with manageable computational power on edge hardware (e.g., NVIDIA Jetson).
Retail Analytics & Automated Checkout	**Faster R-CNN / YOLOv8 Medium**	Prioritizes fine-grained classification accuracy over ultra-low latency to correctly recognize overlapping or tightly packed items on shelves.
Industrial Quality Inspection	**Faster R-CNN / YOLOv8 Large**	Requires dense feature extraction to detect sub-millimeter defects on fast-moving assembly lines, where false discovery rates must be minimized.

Step 10: Summary of Observations & Discussion

Experimental Observations

- **Simultaneous Tasks:** Deep learning object detectors execute spatial bounding box regression ($[x_{\min}, y_{\min}, x_{\max}, y_{\max}]$) and multi-class confidence scoring simultaneously in a single forward pass.
- **Accuracy vs. Latency Trade-off:** Lightweight architectures like **YOLOv8 Nano** achieve low inference latency, ideal for real-time edge execution. Medium variants (**YOLOv8 Small**) capture finer spatial features and occluded objects at a slight latency trade-off.

Advantages of Deep Learning-Based Object Detection

1. **End-to-End Feature Learning:** Replaces manual feature extraction methods (e.g., SIFT, HOG, Haar Cascades) with learned deep convolutional features.
2. **Real-Time Capability:** Single-shot detectors process full frames in one pass, enabling video-stream analytics.
3. **High Generalization:** Pre-trained weights on standard benchmarks (e.g., MS COCO) transfer well to varied lighting, scale, and background conditions.

Limitations

1. **Hardware Dependence:** High-speed inference and training require specialized hardware acceleration (GPU/NPU).
2. **Occlusion & Small Object Challenges:** Downsampling through deeper convolutional layers can lead to spatial feature loss for dense or small objects.
3. **Labeling Cost:** Creating bounding box annotations for custom domain training remains labor-intensive.